

[CS230] Foundations of Neural Networks: Broadcasting & Removing Loops

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1. Deep Learning Big Picture - *Completed*

Chapter 2. Logistic Regression as a Neural Network

- 2.1 Architecture & Forward Propagation - *Completed*
- 2.2 Cost Function & Gradient Descent - *Completed*
- 2.3 Python & Vectorization - *Completed*
- **2.4 Broadcasting & Removing Loops (Current Unit)**
 - * Definition & Rules
 - * Visual Analogy
 - * Under the Hood (Strides)
 - * Implementation (Normalization)

Chapter 3. Shallow Neural Networks - *Upcoming*

지난 시간 복습 및 연결

지난 강의에서 우리는 딥러닝 속도의 핵심인 벡터화(Vectorization)를 배웠습니다. 벡터화가 고속도로(Engine)라면, 오늘 배울 브로드캐스팅(Broadcasting)은 차선을 자유자재로 변경하는 유연함(Flexibility)입니다. 많은 학생들이 'np.dot'은 잘 쓰면서도, 모양이 다른 행렬끼리 연산할 때 발생하는 오류에는 속수무책입니다. 이 원리를 알아야 진정한 디버깅 마스터가 될 수 있습니다.

1 Unit Overview

□ 핵심 목표

이 단원은 서로 다른 모양(Shape)의 데이터를 오류 없이 연산하는 방법을 다룹니다.

- **개념:** NumPy가 작은 배열을 자동으로 확장(Stretch)하여 연산하는 규칙을 배웁니다.
- **구현:** 'for-loop' 없이 데이터 정규화(Normalization)를 수행하는 코드를 작성합니다.
- **원리:** 메모리 복사 없이 스트라이드(Strides) 조작을 통해 효율적으로 동작하는 내부 원리를 이해합니다.

2 Essential Terminology

용어	설명	한 줄 핵심 요약
Broadcasting	브로드캐스팅	모양이 다른 배열 간 연산 시, 작은 쪽을 자동으로 늘려주는 기능
Shape	형상	배열의 차원 크기 (예: (4, 3) 은 4행 3열)
Normalization	정규화	데이터의 평균을 0, 분산을 1로 맞추는 전처리 과정
Keepdims	차원 유지	연산 후에도 차원(Rank)을 삭제하지 않고 유지하는 옵션
Strides	스트라이드	메모리 상에서 다음 요소로 넘어가기 위한 보폭 (Byte 단위)

3 Core Concepts: 브로드캐스팅의 마법

3.1 1. Broadcasting의 정의와 비유

한 줄 요약: 작은 행렬을 큰 행렬 크기에 맞춰 자동으로 '늘려서(Copy)' 연산합니다.

□ 식빵과 버터 비유 (직관적 비유)

식빵 100개(데이터 $m = 100$)에 버터(b)를 발라야 합니다.

- **For-loop:** 식빵을 하나 꺼내고, 버터를 바르고, 내려놓습니다. (100번 반복)
- **Broadcasting:** 마법을 부려 버터를 식빵 100개 길이만큼 순식간에 늘린(Stretch) 뒤, 한 번에 쿡 찍어버립니다.

3.2 2. General Broadcasting Rules (엄격한 규칙)

아무거나 다 늘려주지는 않습니다. NumPy는 뒤(오른쪽) 차원부터 비교하여 다음 조건 중 하나를 만족해야만 연산을 허용합니다.

1. **Equal:** 두 차원의 크기가 같다.
2. **One:** 둘 중 하나의 크기가 1이다. (이 경우 1인 쪽이 늘어남)

Rule Check Example

Case 1: 가능 (Success)

$A : (4, 3)$

$B : (4, 1) \rightarrow 1$ 이 3으로 확장됨.

결과: $(4, 3)$

Case 2: 불가능 (Fail - ValueError)

$A : (4, 3)$

$B : (4, 2) \rightarrow 3$ 과 2는 다르고, 둘 다 1이 아님.

3.3 3. Under the Hood: 가상 복사 (Virtual Copying)

”교수님, 데이터를 늘리면 메모리를 낭비하는 것 아니가요?”

아닙니다. 이것이 브로드캐스팅 기술의 핵심입니다.

- **Physical (실제):** $b = [1, 2, 3]$ (메모리엔 딱 3개만 존재)

- **Logical (가상):** CPU에게는 마치 $[1, 1, 1, \dots], [2, 2, 2, \dots]$ 인 것처럼 주소를 속여서 알려줍니다.
- **Strides Manipulation:** 메모리를 실제로 복사하지 않고, 데이터 접근 보폭(Stride)을 0으로 설정하여 같은 값을 반복해서 읽게 만듭니다. 마치 홀로그램과 같습니다.

4 Numerical Example: 손으로 푸는 브로드캐스팅

□ 칼로리 계산 시나리오 (실전 시나리오 & 계산)

상황: 4가지 음식(행)의 영양소(열: 탄, 단, 지) 데이터가 있습니다. 각 음식의 총 칼로리가 100g당 얼마인지 더하고 싶습니다.

데이터 행렬 A (2개 음식 x 3개 영양소):

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

조미료 B (각 영양소에 추가될 값, 1 x 3):

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

연산 과정 ($A + B$): 행렬 B의 행(Row) 차원이 1이므로, 행렬 A의 크기인 2로 확장됩니다.

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \leftarrow (\text{복사됨}) \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 41 & 52 & 63 \end{bmatrix}$$

5 Implementation: Data Normalization

브로드캐스팅이 가장 빛을 발하는 순간은 데이터를 전처리(Preprocessing) 할 때입니다. 입력 데이터의 평균을 0, 분산을 1로 만드는 정규화를 구현해봅시다.

```
1 import numpy as np
2 import time
3
4 def normalization_demo():
5     # 데이터: 개의4 특성(Feature), 만100 개의 샘플
6     # Shape: (4, 1000000)
7     X = np.random.rand(4, 1000000) * 100
8
9     # 1. 평균과 표준편차 계산
10    # axis=1: 열(column) 방향으로 계산 각(행의 평균)
11    # keepdims=True: (4,)가 아니라 (4, 1)로 유지 -> 브로드캐스팅 필수 조건!
12    mu = np.mean(X, axis=1, keepdims=True)
13    sigma = np.std(X, axis=1, keepdims=True)
14
15    print(f"mu shape: {mu.shape}") # (4, 1) 확인
16
17    # 2. Broadcasting 적용 핵심()
18    # (4, 1000000) - (4, 1) -> (4, 1)이 만100 번 복사되어 연산됨
19    # 나눗셈도 마찬가지로 원리
20    tic = time.time()
21    X_norm = (X - mu) / sigma
22    toc = time.time()
23
24    print(f"Broadcasting time: {1000 * (toc - tic):.2f} ms") # 매우 빠름
25
26 if __name__ == "__main__":
```

Listing 1: Broadcasting Implementation

6 FAQ & Pitfalls

주의: (m

브로드캐스팅의 강력함이 독이 될 때가 있습니다.

- 벡터 A: (5, 1)
- 벡터 B: (1, 5)
- $A + B$: (5, 5) 행렬이 되어버립니다.

두 벡터를 더해서 같은 크기의 벡터를 만들고 싶었다면, 반드시 두 벡터의 Shape이 일치하는지 'assert' 문으로 확인해야 합니다.

Q. keepdims=True를 안 쓰면 어떻게 되나요?

A. 'mu'의 shape이 (4,)가 됩니다. 이를 'Rank-1 Array'라고 합니다. 대부분의 경우 브로드캐스팅이 잘 되지만, 특정 상황에서 예상치 못한 차원 확장이 일어나 디버깅이 매우 어려워집니다. 명시적으로 (4, 1)을 유지하는 것이 안전합니다.

다음 단계 (Next Step)

이제 여러분은 데이터의 모양(Shape)을 자유자재로 다루는 기술까지 익혔습니다. 벡터화와 브로드캐스팅이라는 두 개의 무기를 손에 쥐었습니다.

다음 장 [Chapter 3. Shallow Neural Networks]에서는 드디어 로지스틱 회귀(뉴런 1개)를 넘어서, 은닉층(Hidden Layer)이 있는 진짜 신경망을 구축합니다. 여기서부터 딥러닝의 마법이 시작됩니다.

□ 단원 요약 (Cheat Sheet)

1. **Broadcasting:** 작은 배열을 큰 배열에 맞춰 '가상으로 확장'하여 연산한다.
2. **Rule:** 차원을 오른쪽 끝부터 비교하여, 같거나 1이어야 한다.
3. **Memory:** 데이터를 실제로 복사하지 않으므로 (Strides 조작) 메모리 효율적이다.
4. **Tip:** 'np.sum'이나 'np.mean' 사용 시 'keepdims=True'를 습관화하라.